Part 2 – Srivalli: Data, EDA and the fixed split (Part A)

What my part is

My part is the data foundation of the project. Every model in the team (the scratch CNN, the transfer-learning models, the data-efficiency study and the explainability work) reads images through the code I wrote, so if my part is wrong, every result is wrong. I own three things: `src/dataset.py` (the code that loads the images, makes the split, normalises them, augments them and builds the data loaders), `data/splits.csv` (the saved list that says which image is in train, validation or test), and `notebooks/01_eda.ipynb` (the exploratory data analysis, with the leakage check at the top). The fixed split is the most important output: everybody uses the same 18,900 / 4,050 / 4,050 images, so every team member's score can be compared. The notebook shows what the data looks like (class counts, sample images, colours, histograms), and it checks that there is no leakage between the splits. I also explain why the augmentations (90-degree rotations and flips) are safe for satellite images, and why the test set must never be used while we make decisions.

Definitions

Term	Simple meaning
EuroSAT	A public dataset of satellite image patches, labelled with land-use classes. We use the RGB version.
Sentinel-2	A European Earth-observation satellite. EuroSAT images come from its multispectral sensor.
RGB	Red, Green, Blue. The image has three colour channels, so we keep only the three visible bands.
Tile (patch)	One small image cut from a larger satellite picture. In EuroSAT each tile is 64 x 64 pixels.
Class	One land-use label, for example Forest or River. There are 10 classes.
Label	The class number attached to an image. In our data the labels are 0 to 9.
Split	One of the three parts of the data: train, validation (val) or test.
Train set	The images the model learns from.
Validation set	Images used during development to compare settings and to decide when to stop.
Test set	Images kept back for the final score. They are not used to make decisions.
Stratified split	A split that keeps each class at the same share in every part (for example 70% of each class goes to train).
Data leakage	When information from the test (or validation) data reaches training, so the score looks better than it really is.
Leakage (exact copy)	The same picture appearing in two splits, under the same or a different file name.
Leakage (neighbouring patches)	Two tiles cut from the same large area land in different splits. They look alike, so this can still inflate the score. We could not remove this with a random split.
MD5 hash	A short fingerprint (32 hex characters) computed from a file's bytes. Two files with the same MD5 have the same content.
Augmentation	Making small changes to training images (rotate, flip) so the model sees more variety.
Label-safe augmentation	An augmentation that does not change the true class of the image.
Rotation	Turning the image around its centre. We use only 0, 90, 180 and 270 degrees, so no pixels are blurred.
Flip	Mirroring the image left-right (horizontal) or top-bottom (vertical).

Term	Simple meaning
Normalisation	Shifting and scaling pixel values with a fixed mean and standard deviation, so the numbers are in a similar range for the model.
ImageNet mean and std	The fixed numbers (mean 0.485, 0.456, 0.406 and std 0.229, 0.224, 0.225 for R, G, B) that were measured on the ImageNet photo dataset. We use them for all models so the comparison is fair.
Tensor	A PyTorch array of numbers. The model takes tensors, not PIL images.
PIL image	An image object from the Pillow library. We read files as PIL images, then convert them.
Dataset (PyTorch)	A class that knows how many items there are and returns item number i. Here it returns (image, label).
DataLoader	A PyTorch helper that groups items from a Dataset into batches, can shuffle them and can load them in parallel.
Batch	A group of images that the model processes together. We use 64 images per batch.
Shuffle	Changing the order of the images each epoch, so the model does not see them in the same order every time.
Epoch	One full pass through the training images.
Fraction	The share of the training split we keep. For example 0.05 keeps 5% of train. Val and test never change.
Seed	A number that starts a random generator. The same seed gives the same random choices, so a run can be repeated.
random_state	The scikit-learn name for the seed.
Histogram	A count of how many pixels have each value (0 to 255). It shows how bright or dark, and how spread out, the pixels are.
Per-channel	Separately for each colour channel (red, green, blue).
Mean colour	The average value of the red, green and blue pixels of one class.
Percentile	The value below which a given share of pixels fall. The 5th percentile means 5% of pixels are darker than this value.
Clipping	When bright pixels all go to the maximum value (255), so many pixels pile up at one value.
Class imbalance	Classes that have different numbers of images. Here the ratio is at most 1.5 to 1.
Macro-F1	A score that averages the F1 score of each class equally, so small classes count as much as big ones. We report it with accuracy.
EDA	Exploratory data analysis: looking at the data with counts and plots before training.
assert	A Python statement that stops the program with an error if a condition is false.
pandas DataFrame	A table in memory, with named columns. We use it for the list of images and their splits.
CSV	A plain text table with commas between values.
scikit-learn `train_test_split`	A function that cuts data into two parts, with optional stratification and a seed.
torchvision	The PyTorch library for images. It gives us the EuroSAT dataset and the image transforms.
Lazy loading	Reading an image from disk only when it is asked for, instead of holding all 27,000 in memory.
Ablation	A test where one part is removed, to see how much it mattered.
Data-efficiency study	Training with 5%, 10%, 25%, 50% and 100% of the train data, to see how the score changes with more data.
Reproducibility	Being able to get the same result again from the same code, data and seed.

Part 1: src/dataset.py, function by function

The code below is copied from `src/dataset.py`. Line numbers in the tables refer to that file. The file starts with a docstring (lines 1 to 15) that lists the functions and says how to make the split: `python -m src.dataset`.

1.1 Imports and constants (lines 17 to 32)

```
from pathlib import Path

import pandas as pd
import torch
from PIL import Image
from sklearn.model_selection import train_test_split
from torch.utils.data import DataLoader, Dataset
from torchvision import datasets, transforms

from src.utils import DATA_DIR, SPLITS_CSV

# Team setting: ImageNet numbers for ALL models, so the comparison is fair.
IMAGENET_MEAN = (0.485, 0.456, 0.406)
IMAGENET_STD = (0.229, 0.224, 0.225)

SPLIT_SIZES = {"train": 18900, "val": 4050, "test": 4050}  # 70/15/15 of 27,000
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
17	Imports `Path` from the standard library to handle file paths.	Standard library	`Path` works on Windows and Linux with the same code. Plain strings with `/` or `\` would need care.
19 to 20	Imports pandas (tables) and torch (PyTorch).	Libraries	pandas reads the CSV and does the split bookkeeping. torch is needed for the DataLoader and tensors.
21	Imports `Image` from Pillow to open JPEG files.	PIL image	Pillow is the standard way to read JPEG in Python. `torchvision` also reads PIL.
22	Imports `train_test_split` from scikit-learn.	Stratified split	It has `stratify=` and `random_state=`, which we need. We could write our own split, but that is more code and more chances for bugs.
23	Imports `DataLoader` and `Dataset` from PyTorch.	Dataset and DataLoader	These are the standard PyTorch classes that training loops expect.
24	Imports `datasets` and `transforms` from torchvision.	torchvision	`datasets.EuroSAT` gives us the images and labels. `transforms` gives us rotations, flips, `ToTensor` and `Normalize`.
26	Imports the shared paths `DATA_DIR` and `SPLITS_CSV` from `src/utils.py`.	Single source of paths	Nobody types a path by hand. `DATA_DIR` is `ROOT / "data"` and `SPLITS_CSV` is `DATA_DIR / "splits.csv"`. The repo folder name has spaces, so this avoids path mistakes.
29 to 30	Stores the ImageNet mean and std for R, G, B.	Normalisation	The team uses one setting for all models. Pretrained models expect these numbers. We could compute the mean and std of EuroSAT, but then the input would differ from the pretrained models.
32	Stores the expected sizes 18,900 / 4,050 / 4,050.	Expected numbers	`check_split` compares against these. If the split changes by accident, the check fails.

1.2 `load_eurosat(root=DATA_DIR)` (lines 35 to 38)

```
def load_eurosat(root=DATA_DIR):
    """Load the EuroSAT images that check_setup.py downloaded into data/eurosat/2750."""
    # .samples is a list of (file path, label) pairs, .classes is the list of class names
    return datasets.EuroSAT(root=str(root), download=False)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
35	Defines the function. `root` defaults to the `data` folder.	Default argument	The caller can point somewhere else for tests, but normally nothing needs to be passed.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
38	Opens EuroSAT from `data/eurosat/2750` without downloading. `root=str(root)` turns the Path into text.	torchvision dataset, `download=False`	`download=False` means the code never downloads 94 MB by surprise. The download is done once by `check_setup.py --download`. We could list the folders by hand, but torchvision already gives `.samples` (list of path and label) and `.classes` (the class names).

The object returned has `.samples` (27,000 pairs of file path and label) and `.classes` (the 10 names in alphabetical order). Label 0 is AnnualCrop, and the order follows the folder names.

1.3 `make_split(seed=42)` (lines 41 to 67)

```
def make_split(seed=42):
    """Make the fixed 70/15/15 split and save it to data/splits.csv."""
    eurosat = load_eurosat()

    # One row per image. We save the path from the data folder with "/" so the
    # same file works on every laptop (the full path is different on each one).
    rows = []
    for path, label in eurosat.samples:
        relative_path = Path(path).relative_to(DATA_DIR).as_posix()
        rows.append((relative_path, label, eurosat.classes[label]))
    df = pd.DataFrame(rows, columns=["path", "label", "class_name"])

    # stratify= keeps the same share of each class in every part.
    # First cut: 70% train, 30% rest.
    train_idx, rest_idx = train_test_split(
        df.index, train_size=0.7, stratify=df["label"], random_state=seed)
    # Second cut: half of the rest is validation, half is test (15% + 15%).
    val_idx, test_idx = train_test_split(
        rest_idx, train_size=0.5, stratify=df.loc[rest_idx, "label"], random_state=seed)

    df.loc[train_idx, "split"] = "train"
    df.loc[val_idx, "split"] = "val"
    df.loc[test_idx, "split"] = "test"

    # lineterminator: same line endings on every laptop, so git sees no fake changes
    df.to_csv(SPLITS_CSV, index=False, lineterminator="\n")
    return df
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
43	Opens the EuroSAT data.	Loading	Same loader as 1.2.
47 to 52	Builds one row per image: relative path, integer label and class name.	Table of images	`Path(path).relative_to(DATA_DIR)` cuts off the laptop-specific start, so the path is `eurosat/2750/...`. `.as_posix()` forces `/`. Without this the file would only work on one computer.
51	Turns the rows into a pandas DataFrame with the column names `path`, `label`, `class_name`.	DataFrame	Easy to filter, count and save.
55 to 56	First cut: 70% of the row numbers go to `train_idx`, 30% go to `rest_idx`. `stratify=df["label"]` keeps the class shares. `random_state=seed` (42) makes it repeatable.	Stratified split, seed	Without `stratify`, random chance could put too few images of one class in a part. Without a fixed seed, every run would make a different split.
58 to 59	Second cut: the 30% rest is cut in half. Half is validation, half is test, so each is 15% of the whole data. The labels used for stratifying are taken from the `rest_idx` rows only.	Three-way split	scikit-learn only cuts into two parts, so we cut twice. The stratify labels must match the rows being cut (`df.loc[rest_idx, "label"]`), otherwise the rows and labels would not line up.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
61 to 63	Writes `train`, `val` or `test` into a new `split` column for each row.	pandas `.loc`	Each image gets exactly one value. A row cannot be in two splits, because the column holds one value per row.
66	Saves the table as `data/splits.csv` with no index column. `lineterminator="\n"` asks for Unix line endings.	Reproducibility, git	Everyone uses the same committed file. See the note on line endings in section 4.2.
67	Returns the DataFrame.	Return value	`__main__` passes it straight to `check_split`.

1.4 `check_split(df)` (lines 70 to 89)

```
def check_split(df):
    """Prove there is no leakage. Stops with an error if something is wrong."""
    # 1. The split sizes are the expected ones.
    sizes = df["split"].value_counts().to_dict()
    assert sizes == SPLIT_SIZES, f"wrong split sizes: {sizes}"

    # 2. No file appears twice, so no file can be in two splits.
    assert df["path"].is_unique, "a file appears in more than one row"

    # 3. Every class has the same share in each split (at most 1 image off from exact rounding).
    counts = pd.crosstab(df["class_name"], df["split"])
    total_per_class = counts.sum(axis=1)
    for split, size in SPLIT_SIZES.items():
        expected = total_per_class * size / len(df)
        assert ((counts[split] - expected).abs() <= 1).all(), f"{split} is not stratified"

    print("Leakage check: OK")
    print(f"  split sizes: {sizes}")
    print("  no file appears in two splits; every class keeps its share in every split")
    return counts
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
73	Counts the rows per split and turns the result into a dictionary.	`value_counts`	Simple way to see the sizes.
74	Stops with an error if the sizes are not exactly 18,900 / 4,050 / 4,050. The message shows the real sizes.	`assert`, expected numbers	A wrong split should fail loudly. The check compares the dictionaries, so the order of the keys does not matter.
77	Stops if any `path` appears twice.	Data leakage (exact copy by name)	Each file has one row and one split. A repeated path would mean one file in two splits.
80	Builds a table: rows are classes, columns are splits, values are image counts.	`pd.crosstab`	Shows the class balance in one table, which is also printed in the notebook.
81	Adds up each class's total across the three splits.	Class totals	Needed to compute the expected share.
82 to 84	For each split, computes the expected count per class (class total times split size divided by 27,000) and checks that the real count is at most 1 away.	Stratification check, rounding	The check allows 1 image of difference because a count must be a whole number. In our data every value is exact (for example 2,000 x 18,900 / 27,000 = 1,400). We could require exact equality, but that would fail on other data sizes.
86 to 88	Prints the result: OK, the sizes and the two plain-English claims.	Output	The notebook and the `__main__` block show these lines.
89	Returns the crosstab.	Return value	Notebook cell 3 displays it.

Note: this function checks the file names and the counts only. It does not compare the bytes of the images. That is done by the MD5 check in notebook cell 5 (section 2.3).

1.5 `augment()` (lines 92 to 101)

```
def augment():
    """Random 90-degree rotation and flips. Only used on training images."""
    # A satellite looks straight down, so there is no "up": a rotated or flipped
    # field is still a field. The angles are exact, so no pixels get blurred.
    return transforms.Compose([
        transforms.RandomChoice([transforms.RandomRotation((angle, angle))
                                 for angle in (0, 90, 180, 270)]),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomVerticalFlip(p=0.5),
    ])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
96	`Compose` runs the steps in order: first rotation, then horizontal flip, then vertical flip.	Transform pipeline	Each step is applied to the output of the previous one.
97 to 98	`RandomChoice` picks one of four rotation objects: 0, 90, 180 or 270 degrees. `(angle, angle)` means the range from angle to the same angle, so the rotation is exact.	Rotation, label-safe augmentation	If we wrote `RandomRotation(90)`, torchvision would pick any angle between -90 and +90 degrees, for example 37 degrees. That blurs the pixels and leaves dark empty corners. Exact right angles only move pixels.
99	`RandomHorizontalFlip(p=0.5)` flips left-right with a 50% chance.	Flip	With the rotations, this gives all 8 orientations of a square (4 rotations times 2 flips).
100	`RandomVerticalFlip(p=0.5)` flips top-bottom with a 50% chance.	Flip	Same as above.

The notebook (cell 20) checks that all 8 orientations really occur and that the pixel values do not change. Note that the flips are random, so one image can get any of the 8 orientations on different epochs.

1.6 `make_transform(use_augment=False)` (lines 104 to 108)

```
def make_transform(use_augment=False):
    """Turn a PIL image into a normalised tensor, with augmentation if asked."""
    steps = [augment()] if use_augment else []
    steps += [transforms.ToTensor(), transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD)]
    return transforms.Compose(steps)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
106	If augmentation is asked for, the list starts with `augment()`. Otherwise the list is empty.	Conditional pipeline	Validation and test use `use_augment=False`, so they are never augmented.
107	Adds `ToTensor()` and `Normalize(...)` to the list.	Normalisation, tensor	`ToTensor()` turns a PIL image of 0 to 255 integers into a float tensor of 0.0 to 1.0, in the order channel x height x width. `Normalize` then computes `(x - mean) / std` for each channel.
108	Returns the pipeline.	Return value	The Dataset calls this on every image it reads.

The augmentation runs before `ToTensor()` because it works on the PIL image. Rotating a tensor would also work, but the PIL version is the standard path.

1.7 `EuroSATDataset` (lines 111 to 125)

```
class EuroSATDataset(Dataset):
    """One part of the fixed split: gives back (image, label)."""

    def __init__(self, split, transform=None):
        df = pd.read_csv(SPLITS_CSV)
        self.df = df[df["split"] == split].reset_index(drop=True)
        self.transform = transform if transform is not None else make_transform()

    def __len__(self):
```

```
        return len(self.df)

    def __getitem__(self, i):
        row = self.df.iloc[i]
        image = Image.open(DATA_DIR / row["path"]).convert("RGB")
        return self.transform(image), int(row["label"])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
111	The class is a PyTorch `Dataset`.	Dataset	Any PyTorch `DataLoader` can use it.
114 to 115	`__init__` reads `splits.csv` and keeps only the rows of one split (`"train"`, `"val"` or `"test"`).	Fixed split	The file is the single source of truth, so every model reads the same rows.
116	`reset_index(drop=True)` renumbers the kept rows 0, 1, 2, ...	Index	`__getitem__(i)` uses `iloc[i]`, so the numbers must have no gaps.
117	Stores the transform. If none is given, it uses `make_transform()` (no augmentation, ImageNet normalisation).	Default argument	A safe default: no augmentation unless someone asks.
120	Returns the number of images in this split.	`__len__` rule	The DataLoader needs the length to plan batches. For train it gives 18,900.
123	Reads the row for item `i`.	`iloc`	Row lookup in pandas. It is a little slow for large data, but fine for 27,000 small rows. A faster design would keep a NumPy array of paths.
124	Opens the image from disk and converts it to RGB. `DATA_DIR / row["path"]` builds the full path on this laptop.	Lazy loading, PIL	Only the images in the current batch are in memory. `convert("RGB")` makes sure there are three channels, even if a file were grey.
125	Returns `(image, label)`. The label is turned into a plain Python `int`.	Dataset rule	This is what a training loop expects. The `int()` avoids a pandas number type reaching PyTorch.

1.8 `get_loader(split, fraction, seed, augment, batch_size, num_workers)` (lines 128 to 149)

```
def get_loader(split, fraction=1.0, seed=0, augment=False, batch_size=64, num_workers=0):
    """Make the DataLoader for "train", "val" or "test".

    fraction: share of the TRAIN split to use (stratified). val and test never shrink.
    seed:     picks the subset and the shuffle order, so a run can be repeated.
    augment:  rotations and flips, allowed only for the train split.
    """
    if split != "train" and (fraction != 1.0 or augment):
        raise ValueError("fraction and augment are only for the train split")

    data = EuroSATDataset(split, transform=make_transform(augment))

    if fraction < 1.0:
        # Keep the same class shares in the smaller training set.
        keep, _ = train_test_split(
            data.df.index, train_size=fraction, stratify=data.df["label"], random_state=seed)
        data.df = data.df.loc[sorted(keep)].reset_index(drop=True)

    # Only the training data is shuffled. The generator makes the order repeatable.
    generator = torch.Generator().manual_seed(seed)
    return DataLoader(data, batch_size=batch_size, shuffle=(split == "train"),
                      generator=generator, num_workers=num_workers)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
135 to 136	Raises an error if someone asks for a fraction or augmentation on val or test.	Guard against mistakes	Val and test must be the same in every run. A mistake should stop the run, not quietly change the score.
138	Builds the Dataset for the split, with augmentation only if `augment=True`.	Transform choice	Train uses `augment` as asked. Val and test get `augment=False` because the guard above forces it.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
140 to 144	If `fraction < 1.0`, keeps a stratified random subset of the training rows, with the same class shares, and replaces `data.df` with it.	Stratified subset, data efficiency	At 5% a plain random subset could leave a class with very few images. Stratifying keeps the shares. Different seeds give different subsets, which is what the data-efficiency study measures. `sorted(keep)` puts the rows back in file order, so the result does not depend on how scikit-learn shuffled them.
147	Makes a random generator with a fixed seed.	Seed, generator	Only the DataLoader's shuffle uses this generator, so the batch order is repeatable.
148 to 149	Builds the DataLoader. `shuffle=(split == "train")` shuffles train only. `batch_size` is 64 by default. `num_workers=0` loads in the main process.	Mini-batch training	Shuffling helps learning. Val and test are not shuffled, so the order always matches the file order.

Two details to know. First, the parameter named `augment` shadows the function `augment()` inside `get_loader`. This works, because `get_loader` only passes the boolean on to `make_transform(augment)`, but it is a naming trap. Second, the random augmentations use PyTorch's global random number generator, not the `generator` object. So repeatability of the augmentations depends on `set_seed` from `src/utils.py` (or on the code calling it), not on the `seed` argument of `get_loader`. The `seed` argument only fixes the subset and the shuffle order.

1.9 The `__main__` block (lines 152 to 154)

```
if __name__ == "__main__":
    # Make data/splits.csv and check it.
    check_split(make_split())
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
152	Runs only when the file is started directly, for example `python -m src.dataset`. It does not run when another file imports it.	`__main__`	Importing the module must not rewrite `splits.csv`.
154	Makes the split, saves it, then checks it.	Make and check	One command makes the file and proves it is correct.

Part 2: notebooks/01_eda.ipynb, step by step

The notebook must run from the repo, with `python -m src.dataset` run once before it. The colours and histograms use train images only, so the test set is not used to draw conclusions. Cell numbers below are the notebook cell indexes (0 is the first cell). Markdown cells are summarised; code cells are copied.

2.1 Cell 0 and cell 1: title and setup

Cell 0 is a markdown title with the owner (Srivalli), the part (A) and a reminder to run `python -m src.dataset` first.

```
import hashlib
import sys
from pathlib import Path

sys.path.insert(0, str(Path.cwd().parent))  # the notebook runs in notebooks/, our code is one folder up

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from PIL import Image

from src.dataset import augment, check_split, get_loader
```

```
from src.utils import DATA_DIR, FIGURES_DIR, SPLITS_CSV

FIGURES_DIR.mkdir(parents=True, exist_ok=True)
df = pd.read_csv(SPLITS_CSV)
train = df[df["split"] == "train"]
class_names = sorted(df["class_name"].unique())
print(len(df), "images,", len(class_names), "classes")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1 to 3	Imports `hashlib` (MD5), `sys` and `Path`.	Standard library	Needed for the fingerprint, the path to the code and the figure folder.
5	Adds the repo folder (one level up from `notebooks/`) to Python's import path.	Import path	So `from src.dataset import ...` works from inside `notebooks/`.
7 to 10	Imports plotting and data libraries.	matplotlib, numpy, pandas, Pillow	matplotlib draws the plots, numpy does the histogram maths, pandas holds the table, Pillow opens images.
12	Imports our own functions and paths.	Reuse	The notebook uses the same `check_split`, `augment` and `get_loader` as training, so it shows what the models see.
14	Creates the figure folder if it does not exist.	Folder setup	`parents=True` creates missing parents, `exist_ok=True` avoids an error if it already exists.
15 to 17	Reads the CSV, keeps the train rows, and sorts the class names.	Sorted class order	Sorting gives the same order in every plot.
18	Prints the total and the number of classes.	Sanity check	Output: `27000 images, 10 classes`.

2.2 Cell 3: leakage check (section 1 of the notebook)

```
counts = check_split(df)
counts[["train", "val", "test"]]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Runs the same check as the `__main__` block and keeps the crosstab.	Leakage check	The notebook prints `Leakage check: OK` before any plot, as the markdown asks.
2	Displays the per-class table.	Table output	The reader sees the class shares directly.

Output: `Leakage check: OK`, `split sizes: {'train': 18900, 'val': 4050, 'test': 4050}`, and the table in section 8 below.

2.3 Cell 5: MD5 duplicate check

```
# Same fingerprint = exactly the same file content.
df["md5"] = [hashlib.md5((DATA_DIR / p).read_bytes()).hexdigest() for p in df["path"]]

splits_per_copy = df.groupby("md5")["split"].nunique()
same_content_across_splits = int((splits_per_copy > 1).sum())
print("identical pictures found in more than one split:", same_content_across_splits)
print("Content check:", "OK" if same_content_across_splits == 0 else "NOT OK, look at these files")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
2	For every one of the 27,000 files, reads all its bytes and computes an MD5 hex string.	MD5 hash, fingerprint	The file name check in `check_split` cannot see two copies saved under different names. The fingerprint can. Reading bytes is slow but takes a short time for 27,000 small JPEGs.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
4	For each fingerprint, counts how many different splits it appears in.	`groupby`, `nunique`	If an MD5 appears in two splits, the same picture is in two splits.
5	Counts the fingerprints that appear in more than one split.	Leakage count	The result must be 0.
6 to 7	Prints the count and `Content check: OK` when it is 0.	Output	The notebook shows `identical pictures found in more than one split: 0`.

Limit of this check: it finds identical bytes only. Two JPEG files with the same picture but a different encoding would have different MD5s. It also cannot see that two different tiles overlap or come from the same large scene. That is why the markdown says it "does not rule out neighbouring patches".

2.4 Cell 8: class counts bar chart (section 2 of the notebook)

```
class_counts = df["class_name"].value_counts().reindex(class_names)

fig, ax = plt.subplots(figsize=(9, 4))
ax.bar(class_names, class_counts)
ax.set_ylabel("number of images")
ax.set_title("Images per class (all 27,000)")
plt.setp(ax.get_xticklabels(), rotation=40, ha="right")
fig.tight_layout()
fig.savefig(FIGURES_DIR / "eda_class_counts.png", dpi=120)
plt.show()
print(class_counts.to_string())
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Counts images per class and puts them in sorted class order.	Class count	The bar order matches the other plots.
3 to 4	Creates a figure 9 by 4 inches with one axis.	matplotlib	Standard single-plot setup.
5 to 6	Draws the bars and labels the y-axis and title.	Bar chart	A bar chart is the simplest way to show imbalance.
7	Turns the class names on the x-axis by 40 degrees so they fit.	Layout	Long names like HerbaceousVegetation would overlap otherwise.
8 to 9	Fits the layout, saves the PNG at 120 dpi, and shows it.	Figure output	Saved for the report (`eda_class_counts.png`).
10	Prints the counts as text.	Numbers	The text version is what the markdown cites.

Reading: five classes have 3,000 images, four have 2,500, Pasture has 2,000. The ratio is 1.5 to 1 at most.

2.5 Cell 11: five sample images per class

```
fig, axes = plt.subplots(len(class_names), 5, figsize=(7, 14))
for row, name in enumerate(class_names):
    samples = train[train["class_name"] == name].sample(5, random_state=42)
    for col, path in enumerate(samples["path"]):
        axes[row, col].imshow(Image.open(DATA_DIR / path))
        axes[row, col].set_xticks([])
        axes[row, col].set_yticks([])
    axes[row, 0].set_ylabel(name, rotation=0, ha="right", va="center", fontsize=9)
fig.tight_layout()
fig.savefig(FIGURES_DIR / "eda_samples.png", dpi=120)
plt.show()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Makes a grid with 10 rows (one per class) and 5 columns.	Grid of subplots	Shows five examples of every class at once.
3	Loops over the classes.	Loop	Each row is one class.
4	Picks 5 random train images of this class with `random_state=42`.	Fixed random sample	The seed means the same five images every time, and they are chosen by chance, not by hand.
5 to 6	Opens each image and draws it in its cell.	Image display	Shows the real 64 x 64 tiles.
7 to 8	Hides the tick marks.	Clean display	Ticks would only add noise to an image.
9	Writes the class name on the left of each row.	Label	Lets the reader match rows to classes.
10 to 12	Fits the layout, saves `eda_samples.png` and shows it.	Output	Figure for the report.

Reading: Forest and SeaLake are smooth and dark. Industrial and Residential have many small bright details. Highway tiles contain fields and buildings too, so one tile can hold more than its label says.

2.6 Cell 14: per-class mean colour (section 4 of the notebook)

```
# Count how many pixels have each value 0..255, per class and per colour channel.
# We keep these counts for the mean colour here and the histograms in section 5.
hist = {name: np.zeros((3, 256), dtype=np.int64) for name in class_names}
for path, name in zip(train["path"], train["class_name"]):
    pixels = np.asarray(Image.open(DATA_DIR / path).convert("RGB"))
    for channel in range(3):
        hist[name][channel] += np.bincount(pixels[:, :, channel].ravel(), minlength=256)

values = np.arange(256)
mean_colour = pd.DataFrame(
    {name: (hist[name] * values).sum(axis=1) / hist[name].sum(axis=1) for name in class_names},
    index=["R", "G", "B"]).T

fig, ax = plt.subplots(figsize=(9, 3))
ax.imshow([mean_colour.to_numpy() / 255], aspect="auto")
ax.set_xticks(range(len(class_names)), class_names, rotation=40, ha="right")
ax.set_yticks([])
ax.set_title("Mean colour of each class (train images, true scale)")
fig.tight_layout()
fig.savefig(FIGURES_DIR / "eda_mean_colour.png", dpi=120)
plt.show()
mean_colour.round(1)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
4	Makes one array of 3 x 256 zeros per class. Row 0 is red, row 1 green, row 2 blue. Each column is a pixel value 0 to 255.	Histogram, per-channel	The counts can be added up across images without keeping the pixels.
5 to 7	For each train image, opens it and, for each of the three channels, adds a count of each pixel value.	`np.bincount`, per-channel histogram	Counting is much lighter on memory than storing 18,900 images. `ravel()` flattens the 64 x 64 channel to one line of numbers.
10	Makes the array `0, 1, ..., 255`.	Weights	Used to turn counts into a sum of values.
11 to 13	Mean per channel: (count times value), summed over 0 to 255, divided by the total count. Put into a table with rows = classes and columns R, G, B.	Mean colour	This is the average pixel value per channel. It is exact, because it uses all pixels of all train images.
15 to 16	Makes a wide, short figure.	Figure	One strip of colours.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
17	Draws the mean colours as a single image strip, divided by 255 so the values are between 0 and 1.	Colour display	Shows the colours at true scale, so the reader sees how dark or bright each class is.
18 to 21	Puts class names on the x-axis, removes the y-ticks, sets the title, saves and shows the figure.	Output	Saved as `eda_mean_colour.png`.
22	Displays the table, rounded to one decimal.	Numbers	This is the table quoted in the report.

Reading: Forest (39.4, 64.5, 75.9) is darkest and Industrial (123.4, 125.5, 134.4) brightest. AnnualCrop and PermanentCrop are close, and Pasture and River are close. Colour alone cannot separate all classes.

2.7 Cell 17: per-channel histograms and percentiles (section 5 of the notebook)

```
fig, axes = plt.subplots(2, 5, figsize=(14, 5), sharex=True, sharey=True)
for ax, name in zip(axes.ravel(), class_names):
    for channel, colour in enumerate(["red", "green", "blue"]):
        share = hist[name][channel] / hist[name][channel].sum()
        ax.plot(values, share, color=colour, linewidth=1)
    ax.set_title(name, fontsize=10)
for ax in axes[1]:
    ax.set_xlabel("pixel value")
for ax in axes[:, 0]:
    ax.set_ylabel("share of pixels")
axes[0, 0].set_ylim(0, 0.08)  # SeaLake and Forest have tall peaks; the same scale keeps classes comparable
fig.tight_layout()
fig.savefig(FIGURES_DIR / "eda_histograms.png", dpi=120)
plt.show()

# Numbers to back up what we read from the plot: the middle value and the spread of each class.
rows = {}
for name in class_names:
    grey = hist[name].sum(axis=0)
    cumulative = np.cumsum(grey) / grey.sum()
    rows[name] = {"p5": int(np.searchsorted(cumulative, 0.05)),
                  "median": int(np.searchsorted(cumulative, 0.50)),
                  "p95": int(np.searchsorted(cumulative, 0.95))}
pd.DataFrame(rows).T
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Makes a 2 by 5 grid, one small plot per class, all with the same axis scale.	Shared axes	Same scale lets the reader compare classes by eye.
2 to 5	For each class, draws three lines (red, green, blue). Each line is the share of pixels with that value.	Normalised histogram	Dividing by the total turns counts into shares, so classes with different numbers of pixels are comparable.
6	Sets the title to the class name.	Label	Lets the reader read each plot.
7 to 10	Puts x-labels on the bottom row and y-labels on the first column.	Layout	Less clutter than labelling every plot.
11	Limits the y-axis to 0 to 0.08.	Scale choice	SeaLake and Forest have tall peaks. The limit keeps the others readable. Peaks above the limit are cut off in the plot, which is a known trade-off.
12 to 14	Fits the layout, saves `eda_histograms.png`, shows it.	Output	Figure for the report.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
18 to 19	Adds the three channel histograms for one class into one grey histogram.	Pooled histogram	The percentiles use all channels together, so one number describes the brightness of the class.
20	Turns the counts into a cumulative share from 0 to 1.	Cumulative distribution	The value where the cumulative share reaches 0.05 is the 5th percentile.
21 to 23	Finds the first pixel value where the cumulative share reaches 0.05, 0.50 and 0.95.	`searchsorted`, percentile	These are the 5th percentile, median and 95th percentile. They turn the plot into numbers.
24	Shows the table with one row per class.	Numbers	Output in the notebook.

Reading: Forest's 5% to 95% range is 33 to 84. AnnualCrop is 60 to 202, Industrial 66 to 243. AnnualCrop and Industrial show a spike at 255 (clipping). SeaLake has the lowest 5th percentile (24). In most classes red is lower than green and blue.

2.8 Cell 20: augmentation demo and exact-pixel check (section 6 of the notebook)

```
image = Image.open(DATA_DIR / train[train["class_name"] == "Highway"]["path"].iloc[0]).convert("RGB")
transform = augment()

fig, axes = plt.subplots(1, 8, figsize=(12, 1.8))
for ax in axes:
    ax.imshow(transform(image))
    ax.axis("off")
fig.savefig(FIGURES_DIR / "eda_augment.png", dpi=120, bbox_inches="tight")
plt.show()

# A rotation or flip only moves pixels, it must not change any value.
original = np.sort(np.asarray(image).ravel())
same_pixels = all(np.array_equal(original, np.sort(np.asarray(transform(image)).ravel())) for _ in range(50))
print("augment keeps the exact pixel values:", same_pixels)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Opens the first Highway image in the train split.	Fixed example	Always the same image, so the figure is repeatable.
2	Builds the augmentation pipeline from `augment()`.	Augmentation	Same pipeline as training.
4 to 8	Draws eight random augmented versions of the image in a row.	Visual check	The reader can see the rotations and flips. Each run gives different random versions.
9	Saves `eda_augment.png` with tight edges.	Output	Figure for the report.
12	Sorts all pixel values of the original, so order does not matter.	Value check	Rotations and flips only move pixels, so the sorted list of values is the same.
13	Repeats 50 random augmentations and checks that the sorted pixel values are identical every time.	Exact check	`True` means no pixel was blurred, interpolated or lost.
14	Prints `augment keeps the exact pixel values: True`.	Output	Shown in the notebook.

2.9 Cell 21: loader check and fraction sizes

```
images, labels = next(iter(get_loader("train", seed=0, augment=True)))
print("batch:", images.shape, labels.shape)
print("mean after normalising:", round(images.mean().item(), 3), " std:", round(images.std().item(), 3))

for fraction in [0.05, 0.1, 0.25, 0.5, 1.0]:
    subset = get_loader("train", fraction=fraction, seed=0).dataset.df
    print(f"fraction {fraction:<5} {len(subset):>6} images, smallest class {subset['label'].value_counts().min()}")

print("val:", len(get_loader("val").dataset), " test:", len(get_loader("test").dataset))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Builds the train DataLoader with augmentation and takes its first batch.	DataLoader, iterator	Shows exactly what the first training step receives.
2	Prints the shapes: images `[64, 3, 64, 64]` and labels `[64]`.	Batch shape	Batch of 64, 3 channels, 64 x 64 pixels.
3	Prints the mean and std of this batch after normalisation.	Normalisation check	Output: mean -0.161 and std 0.833. Not 0 and 1, because the ImageNet numbers were not computed on EuroSAT. This is a known and accepted difference.
5 to 7	For five fractions, builds the loader and prints how many train images are kept and the size of the smallest class.	Stratified subset	Shows that stratifying keeps every class. At 5% the smallest class still has 70 images.
9	Prints the sizes of val and test.	Fixed sizes	Output: val 4050, test 4050.

Note that the loop builds a full DataLoader for every fraction, which is a little wasteful but harmless for this check.

---

3. The figures in results/figures (eda_*)

Five figures come from my notebook. The other PNG files in that folder belong to other parts (scratch CNN, transfer learning, Grad-CAM, confusion matrices and data efficiency).

File	What it shows	How it was made
`eda_class_counts.png`	Bar chart of the number of images per class (all 27,000). Shows the mild imbalance: 3,000 for five classes, 2,500 for four, 2,000 for Pasture.	Notebook cell 8
`eda_samples.png`	A 10 by 5 grid: five random train tiles per class, with the class name on the left. Shows the look of each class and that one tile can contain more than its label.	Notebook cell 11
`eda_mean_colour.png`	One strip of ten colour blocks, one per class, showing the mean red, green and blue of the train tiles at true scale. Forest is the darkest, Industrial the brightest.	Notebook cell 14
`eda_histograms.png`	A 2 by 5 grid of per-channel histograms (red, green, blue lines) for each class, with the same y-scale up to 0.08. Shows how narrow or wide the brightness spread is, and the spike at 255 for AnnualCrop and Industrial.	Notebook cell 17
`eda_augment.png`	Eight random augmented versions of one Highway tile in a row. Shows that rotations and flips keep the road recognisable.	Notebook cell 20

---

4. data/splits.csv

4.1 Columns and head

The file has four columns:

Column	Meaning
`path`	The image path from the data folder, always with `/`, for example `eurosat/2750/AnnualCrop/AnnualCrop_1.jpg`.
`label`	The class number, 0 to 9, in alphabetical class order.
`class_name`	The class name, for example `AnnualCrop`.
`split`	`train`, `val` or `test`. Each row has exactly one value.

The first rows (after the header) look like this:

```
path,label,class_name,split
eurosat/2750/AnnualCrop/AnnualCrop_1.jpg,0,AnnualCrop,train
eurosat/2750/AnnualCrop/AnnualCrop_10.jpg,0,AnnualCrop,val
eurosat/2750/AnnualCrop/AnnualCrop_100.jpg,0,AnnualCrop,train
```

4.2 Counts

The file has 27,000 data rows plus the header. Counted from the file: train 18,900, val 4,050, test 4,050. There are 27,000 unique paths, so no file is listed twice.

Line endings: the code asks for `\n` (Unix line endings), but on this laptop the file on disk has `\r\n`. The repository has `core.autocrlf` set to `true`, which converts line endings when files are checked out on Windows. This is most likely the reason; it is a harmless difference, but it is worth knowing if someone sees a "changed" file in git after a fresh clone.

---

5. How the split is made, step by step

Command: `python -m src.dataset` (run once, from the repo folder, with the venv on). It runs `check_split(make_split())`.

1. The code opens the EuroSAT RGB data from `data/eurosat/2750` with `download=False`. torchvision finds 27,000 `(path, label)` pairs and 10 class names.
2. A table is built with one row per image: `path` (relative, with `/`), `label` (0 to 9) and `class_name`.
3. First cut with `train_test_split(..., train_size=0.7, stratify=label, random_state=42)`: about 70% of the rows go to train (18,900) and the other 30% to the rest (8,100).
4. Second cut with `train_test_split(rest, train_size=0.5, stratify=label of rest, random_state=42)`: the rest is cut in half. Each half gets 4,050 rows: validation and test.
5. Each row gets its `split` value (`train`, `val` or `test`).
6. The table is saved to `data/splits.csv`, with no index column.
7. `check_split` runs and prints: - `Leakage check: OK` - `split sizes: {'train': 18900, 'val': 4050, 'test': 4050}` - `no file appears in two splits; every class keeps its share in every split`
8. The notebook repeats this check (cell 3) and prints the crosstab of classes by split (section 8).

Because the seed is fixed at 42, running the cut again on the same data gives the same file. This was checked in the work (the split was made twice and the file was the same both times).

---

6. Why the split must be fixed and how we prove no leakage

Why it must be fixed. Every model in the team must be trained on the same train images and scored on the same test images. Otherwise a higher score could come from an easier test set, not a better model. A fixed split, saved in a file that everybody commits, makes the scores comparable. A random split made fresh on each laptop would give five different test sets. The seed (42) and the saved CSV make the split fixed. The CSV is the real source of truth: the code only reads it after the first run.

Why stratified. The classes have different sizes (2,000 to 3,000). A plain random split could put most of one class into train by chance. Stratifying makes each class keep the same 70/15/15 share in every part. This matters most in the data-efficiency study, where at 5% the smallest class has only 70 train images.

How we prove there is no leakage. The checks are:

1. **Sizes.** `check_split` asserts that the split sizes are exactly 18,900, 4,050 and 4,050. Any other number stops the program.
2. **Unique file names.** `check_split` asserts that every `path` appears once. A file cannot be in two splits if it appears only once.

3. **Stratification.** `check_split` builds the class-by-split table and checks that every count is within 1 of the exact expected count. In our data the expected counts are whole numbers, so all counts are exact (for example AnnualCrop has 2,100 / 450 / 450, and Pasture has 1,400 / 300 / 300).
4. **Identical content (MD5).** Notebook cell 5 reads all 27,000 files, computes an MD5 for each and counts the MD5 values that appear in more than one split. The result is 0, so no byte-identical picture is in two splits, even under a different file name.

What the checks do not prove: they do not find neighbouring tiles. EuroSAT tiles are cut from larger scenes, so two tiles from the same area can be in train and test. They look alike, so a model may score higher than on a truly new area. A random split cannot prevent this. The fix would be a split by geographic area, which needs the location of each tile, and EuroSAT does not give that to us in this file. This is stated as a limitation in the report.

---

7. Why rotations and flips are safe here, and why the test set is never touched

7.1 Rotations and flips

A satellite image is taken looking straight down. There is no "up", no sky, and no fixed direction for roads or fields. A field rotated 90 degrees is still a field, and a river flipped left-right is still a river. So the class label does not change. This is why these transforms are label-safe for EuroSAT.

They are also exact. The rotations use only 0, 90, 180 and 270 degrees. A rotation by any other angle would need interpolation, which mixes neighbouring pixels and leaves empty corners. Right-angle moves only change where pixels are; no value is created or lost. Notebook cell 20 checks this: the sorted pixel values of the image are identical after 50 random augmentations.

Together the rotations and the two flips give all 8 orientations of a square tile. The model sees more variety without any new images.

Why this would not work for digits. A handwritten "6" rotated 180 degrees looks like a "9", and a flipped "2" or "5" is not the same digit. The label would change, so rotations and flips are not label-safe for digits. For satellite tiles there is no such confusion, because the class depends on the land cover, not on the direction of the picture.

Why only on training images. Augmentation adds variety for learning. Validation and test must stay as they are, so that every model is scored on the same pictures. The code enforces this in two ways: `make_transform(use_augment=False)` is the default in `EuroSATDataset`, and `get_loader` raises an error if someone asks for augmentation on val or test.

7.2 Why the test set is never touched

The test set is the only honest estimate of how the model will do on new images. It must be used once, at the end, for the final score. If we looked at it while making choices (which model, which learning rate, when to stop), the choices would be tuned to it, and the final score would be too optimistic. This is leakage of a different kind: the test set leaks into the decisions.

In our code and notebook this is handled by four rules:

1. The EDA (colours, histograms, percentiles) uses train images only (cell 14 and cell 17 read `train`, not the whole `df`). The class counts (cell 8) and the leakage checks are about the whole table, but they look only at file names and sizes, not at pixel values, and they are needed to prove the split.
2. Decisions are made on the validation set. Only the training split is shuffled and augmented.
3. `get_loader` refuses `fraction` or `augment` on val or test, so the val and test sets cannot be changed by a setting.
4. The test set is used only in the final evaluation, which is in another part of the project.

8. Numbers you must know

Dataset. EuroSAT RGB: 27,000 tiles, 64 x 64 pixels, 3 colour channels, 10 classes. Sentinel-2 satellite, 10 metres per pixel (from the report's dataset section).

Class counts (whole dataset, and split by 70/15/15).

Class	Total	Train	Val	Test
AnnualCrop	3,000	2,100	450	450
Forest	3,000	2,100	450	450
HerbaceousVegetation	3,000	2,100	450	450
Highway	2,500	1,750	375	375
Industrial	2,500	1,750	375	375
Pasture	2,000	1,400	300	300
PermanentCrop	2,500	1,750	375	375
Residential	3,000	2,100	450	450
River	2,500	1,750	375	375
SeaLake	3,000	2,100	450	450
Total	**27,000**	**18,900**	**4,050**	**4,050**

Imbalance: 3,000 versus 2,000 is a ratio of 1.5 to 1.

Split sizes. 18,900 train, 4,050 validation, 4,050 test. Seed 42. Stratified twice.

Fractions of train used by the data-efficiency study (seed 0).

Fraction	Images	Smallest class
0.05	945	70
0.10	1,890	140
0.25	4,725	350
0.50	9,450	700
1.00	18,900	1,400

Loader. Batch size 64 by default. One batch shape: 64 x 3 x 64 x 64, labels 64.

Normalisation. ImageNet mean (0.485, 0.456, 0.406) and std (0.229, 0.224, 0.225). One augmented training batch after normalising: mean -0.161, std 0.833.

Leakage checks. 27,000 unique paths. MD5 duplicates across splits: 0.

Colour and brightness (train only).

Class	Mean R, G, B	Pixel 5th to 95th percentile
AnnualCrop	129.6, 120.0, 117.2	60 to 202
Forest	39.4, 64.5, 75.9	33 to 84
HerbaceousVegetation	98.0, 98.6, 103.9	57 to 158
Highway	87.8, 96.4, 103.3	44 to 164
Industrial	123.4, 125.5, 134.4	66 to 243

Class	Mean R, G, B	Pixel 5th to 95th percentile
Pasture	62.2, 88.4, 94.2	43 to 127
PermanentCrop	122.1, 115.9, 113.9	68 to 188
Residential	99.3, 103.3, 112.7	62 to 170
River	66.9, 86.6, 96.9	40 to 135
SeaLake	48.0, 72.9, 89.2	24 to 120

Pairs that look alike in colour: AnnualCrop and PermanentCrop; Pasture and River.

Augmentation. 4 rotations (0, 90, 180, 270 degrees) times 2 flips gives 8 orientations.

---

9. Honest notes and open points

- The `augment` name is used for both a function and a parameter of `get_loader`. It works, but it is confusing.
- Augmentation randomness is not controlled by the `seed` argument of `get_loader`. It depends on the global PyTorch seed (set by `set_seed` in `src/utils.py`). A run is repeatable only if that is set as well.
- The leakage check in `check_split` compares file names and counts. The byte-level check is in the notebook, not in the code. A teammate who runs only `python -m src.dataset` does not get the MD5 check.
- The 1-image tolerance in `check_split` is more lenient than needed for this data, where every count is exact.
- The notebook was run on Python 3.13 (metadata in the notebook), while the team standard is Python 3.12. The code should behave the same, but this was not tested on 3.12.
- Neighbouring-tile leakage is not checked and cannot be removed with a random split. It is a stated limitation.